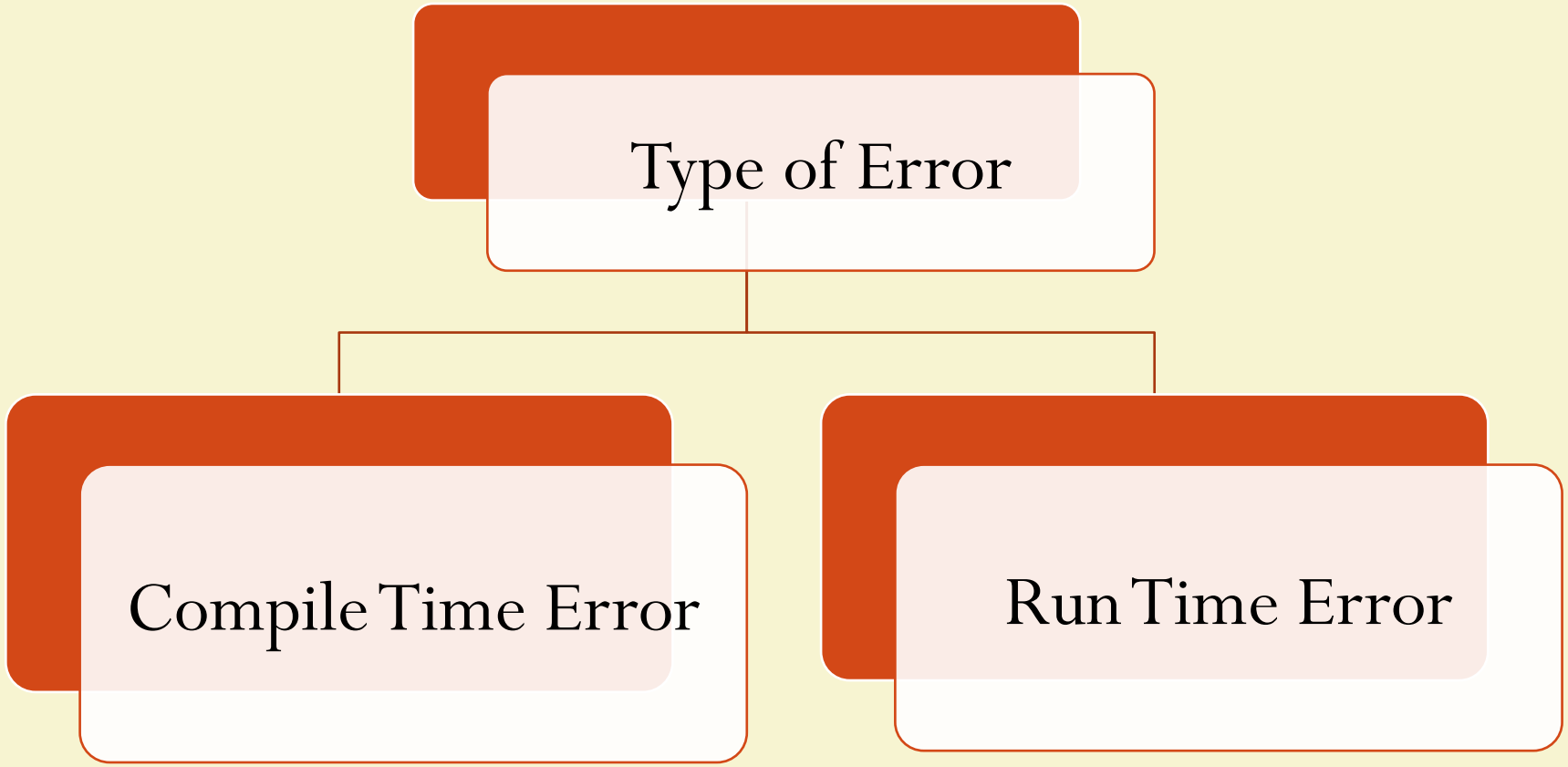
Fundamental of JAVA Programming

UNIT :- V

Exception Handling , Multithreading, Applet

Types of Errors



Compile Time Errors

- All syntax errors will be detected and displayed by the Java compiler ,these errors are called as compile time errors.
- Whenever the compiler displays an error, it will not create the .class file
- It is necessary to fix all these errors before we can successfully compile and run the program

```
class Myclass
{
   public static void main (String args [])
  {
     System.out.println(“Hello Java”)
   }
}
```

Syntax error, insert ";" to complete BlockStatements

Reasons for Compile Time Errors

- Missing Semicolons
- Missing brackets in classes and methods
- Misspelling of identifiers and keywords
- Missing double quotes in string
- Used of undeclared variables
- Bad Reference to objects
- Use of = in place of == operator

Run Time Errors

- Sometime, a program may compile successfully creating the .class file but may not run properly.
- Such programs may produce wrong results due to wrong logic or may terminate due to errors.
- Most Run time Errors are

1. Dividing an Integer by Zero
2. Out of the bounds of an array
3. Store a value into an array of an incompatible type
4. Passing a parameter that is not in a valid range or value for method
5. Attempting negative size of an array
6. Converting invalid string to a number
7. Accessing a character that is out of bounds of a string

Run Time Error Example

```
class Myclass
{
   public static void main (String args [])
  {
     int a=10;
     int b=5;
     int c=5;
     int x=a/(b-c);  //Division by zero
      System.out.println(" x="+x);
      int y=a/(b+c);
     System.out.println("y="+y);
    }
}
```

```
Java.lang.ArithmeticException: /by zero
At Myclass.main(Myclass.java:)
```

Exceptions

- Exception is a condition that is caused by a run time error in the program.
- When the Java interpreter encounters an error such as dividing an integer by zero, it creates an exception object and throws
- If the exception object is not caught and handled properly, the interpreter will display an error message as shown in the output of program and will terminate the program.
- If we want the program to continue with the execution of the remaining code, then we should try to catch the exception object thrown by the error condition and then display an appropriate message for taking corrective actions. This task is known as exception handling.

Exceptions Type

Exception Type	Cause of Exception
ArithmaticException	Caused by math errors such as division by zero
ArrayIndexOutOfBoundException	Caused by bad array indexes
ArrayStoreException	Caused when a program tries to store the wrong type of data in array
FileNotFoundException	Caused by an attempt to access a nonexistent file
IOException	Caused by general I/O failures, such as inability to read from a file
NumberFormatException	Caused when a conversion between strings and numbers fails
OutOfMemoryException	Caused when there's not enough memory to allocate a new object
SecurityException	Caused when an applet tries to perform an action not allowed by the browser security setting
StackOverFlowException	Caused when the system runs out of stack space
StringIndexOutOfBoundException	Caused when a program attempts to access a nonexistent character position in a String

Error Handling

- For Error Handling performs the following tasks

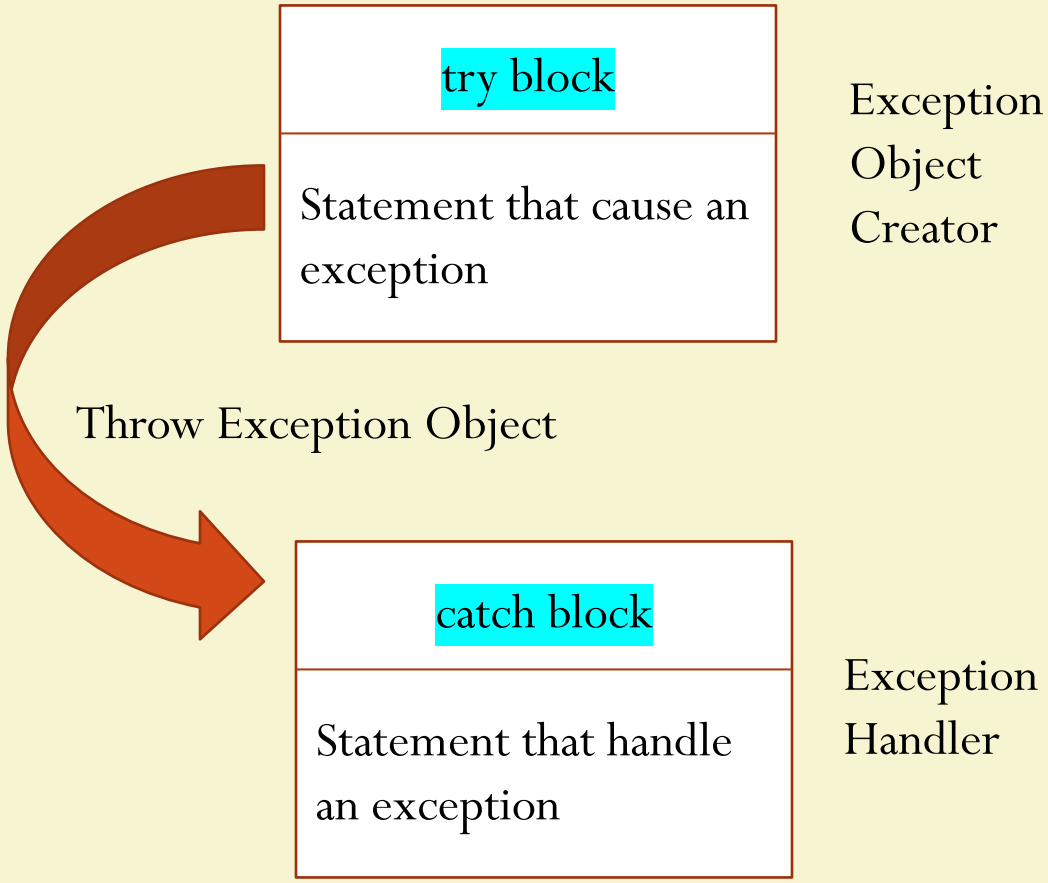
1. Find the problem (Hit the exception)
2. Inform that an error has occurred (throw the exception)
3. Receive the error information (catch the exception)
4. Take corrective actions (handle the exception)

- Java Exception Handling is managed by five keywords

```
try{.....}
catch{.....}
throw
throws
finally{.....}
```

Syntax of Exception Handling Code

```
……..
……..
try
{
   statement;   //Generates an Exception
}
catch (Exception-Type e)
{
   statement ;     //Processes the exception
}
……..
……..
```



Exception Handling In Java

```
class Myclass
{
   public static void main (String args [])
  {
     int a=10;
     int b=5;
     int c=5;
     int x,y;
     try
    {
     x=a/(b-c);  //Exception Here
      System.out.println(" x="+x);
    }
    catch(ArithmeticException e)
    {
      System.out.println("Division by zero"+ "Please take a
      different value of c" );
     }
     int y=a/(b+c);
     System.out.println("y="+y);
}
```

Output:-

Division by zero Please take a different value of c

y=1

Exception Handling In Java

```
class CLineInput
{
   public static void main (String args [])
  {
     int invalid= 0;     //Number of Invalid Argument
     int number, count=0
     for(int i=0;i<args.length;i++)
    {
       try
       {
          number=Integer.parseInt(args[i]);
       }
     catch(NumberFormatException e)
     {
       invalid=invalid +1;   //Caught an Invalid Number
       System.out.println("Invalid Number:- "+args[i]);
       continue;
     }
       count=count+1;
     }    //End of For loop
     System.out.pirntln("Total Valid Number="+count);
     System.out.println("Total Invalid Number="+invalid);
   }
}
```

Java ClineInput 15 25.75 40 Java 10.5 65

Output:-

Invalid Number:- 25.75

Invalid Number:- Java

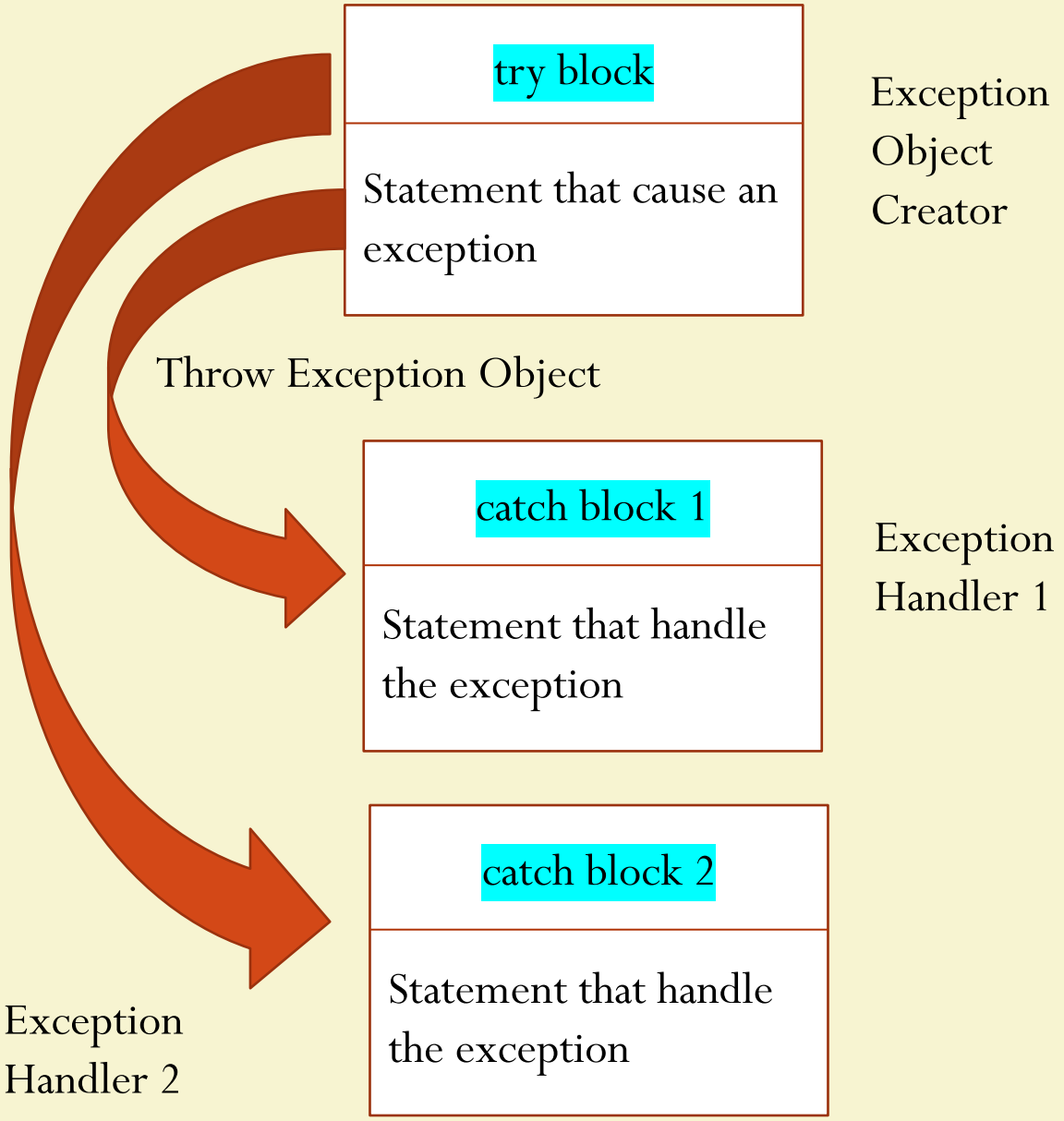
Invalid Number:- 10.5

Total Valid Number:- 3

Total Invalid Number:- 3

Multiple Catch Statement

```
.....
try
{
   statement;   //Generates an Exception
}
catch (Exception-Type-1 e)
{
   statement ;     //Processes the exception type 1
}
catch(Exception-Type-2 e)
{
  statement; //Process the exception type 2
}
.
.
.
catch(Exception Type-N e)
{
    statement;
}
```



Example of Multiple Catch Blocks

```
class MyClass
{
   public static void main (String args [])
  {
     int a[] = {5,10};
     int b=5;
      try
      {
         int x=a[2]/b – a[1];
      }
     catch(ArithmeticException e)
     {
      System.outprintln("Division by zero");
      }
      catch (ArrayIndexOutOfBoundsException e)
      {
         System.out.println("Array Index Error");
      }
catch(ArrayStoreException e)
      {
         System.out.println("Wrong  Data Type");
      }
       int y=a[1]/a[0];
      System.out.println("y="+y);
  }
}
```

Output:-

Array Index Error

y =2

Exception Handling Code with Finally Statement

- Finally statement that can be used to handle an exception that is not caught by any of the previous catch statements.
- Finally block can be used to handle any exception
- It may be added immediately after the try block or after the last catch block
- When finally block is defined, then it is guaranteed to execute, regardless of whether or not an exception is thrown

```
try
{
   statement;   //Generates an Exception
}
finally
{
   statement ;    //Processes the exception
}
.....................
```

```
try
{
   statement;   //Generates an Exception
}
catch (Exception-Type e)
{
   statement ;    //Processes the exception
}
catch (Exception-Type e)
{
   statement ;    //Processes the exception
}
finally
{
   statement ;    //Processes the exception
}
```

Example of Multiple Catch Blocks with Finally

```
class MyClass
{
   public static void main (String args [])
  {
     int a[] = {5,10};
     int b=5;
      try
      {
         int x=a[2]/b – a[1];
      }
     catch(ArithmeticException e)
     {
      System.outprintln("Division by zero");
      }
      catch (ArrayIndexOutOfBoundsException e)
      {
         System.out.println("Array Index Error");
      }
      catch(ArrayStoreException e)
      {
         System.out.println("Array Index Error");
      }
      catch(ArrayStoreException e)
      {
         System.out.println("Wrong  Data Type");
      }
        finally
        {    int y=a[1]/a[0];
             System.out.println("y="+y);
        }
}
```

Output:-

Array Index Error

y =2

Unit No.5, Part I Assignment No.9

- Implement the exception handling using try and catch statements to solve runtime errors.
- Assignment Deadline :-

Fundamental of JAVA Programming

UNIT :- V

Exception Handling , Multithreading, Applet

Topic

Applet

Applet

- Java Programs are available in two flavours
- Application :- It is similar to all other kind of programs like in C , C++ etc. to solve a real time problem
- Applet :- Applet is small Java programs that performs specific task and primarily used in internet computing
- They can be transported over the Internet from one computer to another and run using the Applet viewer or any web browser that supports Java.
- An applet , like any application program, can do many things for us. It can perform arithmetic operations, display graphics, play sounds, accept user input, create animation and play interactive games.
- A web page can now contain not only a simple text or a static image but also a Java applet which run graphics, sounds and moving images.

Local and Remote Applet

- We can embed applets into web pages in two ways
- One , we can write our own applet and embed them into web pages
- Second, we can download an applet from a remote computer system and then embed into a web page.

How Applets Differ from Applications

- Applets and stand alone applications are Java programs
- Significant difference between them:- Applets are not full featured application programs.
- They are usually written to complete a small task or a component of a task.
- They are usually designed for use on the internet

1. Applets do not use the main() method for initiating the execution of the code
2. Unlike stand alone application, applets can not be run independently
3. Applets can not read form or write to the files in the local computer
4. Applets can not communicate with other servers on the network
5. Applets can not run any program from the local computer

Preparing to Write Applets

- Before we try to write applets, we must make sure that Java is installed properly, and also ensure that either the Java appletviewer or a Java enabled browser is available
- Steps involved in developing and testing in applet are

1. Building an applet code (.java file)
2. Creating an executable applet (.class file)
3. Designing a web page using HTML tag
4. Preparing <APPLET>tag
5. Incorporating <APPLET> tag into the web page
6. Testing the applet code

Building Applet Code

- Our applet code use the services of two classes, namely, Applet and Graphics from the Java class library.
- Applet class which is contained in the java.applet package provides life and behaviour of applet through its methods such as init(), start(), paint(), close().
- Therefore Applet class is the life cycle of applet.
- paint () method of the Applet class, when it is called, actually displays the result of the applet code on the screen. Output may be text, graphics, sound etc.
- paint() method, which requires a Graphics object as an argument is defined as

```
public void paint(Graphics g)
```

- This requires that the applet code import the java.awt package that contain the Graphics class.
- All output operation of an applet are performed using the methods defined in the Graphics class.

Building an Applet Code

- Process-1) Import Applet and Graphics class from applet and awt package respectively
- Process -2) Define applet class from extending the properties of Applet class
- Process-3) Override the method paint that shows the result of applet code
- Process -4) By using object of Graphics class access the methods that perform the all operation of applet

- Compile this file to creating a .class file for execution of applet code

```
//File Name is :- AppletClassName.java
import java.applet.Applet;
import java.awt.Graphics;

public class AppletClassName extends Applet
{
    ----------------
     public void paint(Graphics g)
     {
       ----------
       ----------
     }
      --------------
      --------------
}
```

Testing a Applet Code

- We know the applet are programs that reside on web pages. In order to run a Java applet, it is first necessary to have a web page that references that applet.
- Basically web page made up of text and HTML tags that can be interpreted by web browser or an applet viewer.
- A web page also called as HTML page
- A web page is marked by an opening HTML tag <HTML> and a closing HTML tag </HTML> and its divided into three major sections:

1. Comment Section (Optional)
2. Head Section (Optional)
3. Body Section

Testing a Applet Code

<HTML>

```
<!
    ---- -------
    ------------
>
```

Comment Section

```
<HEAD>
        Title Tag
</HEAD>
```

Head Section

```
<BODY>
        Applet Tag
</BODY>
```

Body Section

```
<applet code = "AppletClassName.class"  width ="300" Height="300">
</applet>
```

Save as .html and use appletviewer filename.html commaned to run our applet code

</HTML>

HTML File that hosts an APPLETN

```
<HTML>
  <HEAD>
     <TITLE> A sim
  </HEAD>
  <BODY>
    <APPLET CODE="
    <H1>
       A browser s                                    / it on the screen.
    </H1>
    <B><I>These are some text in Italics and Bold font.</I></B>
```

Note:

1. HTML is case insensitive language.
2. Browser interprets each tag and ignores portion, which is unable to interpret or if there is any error.

Element					
<applet>	Not supported	Not supported	Yes	Yes	Not supported

Note: There is still some support for the <applet> tag in some browsers, but it requires additional plug-ins/installations to work.

Note: The <applet> tag is supported in Internet Explorer 11 and earlier versions, using a plug-in.

The HTML APPLET Tag : Syntax

The syntax for the standard APPLET tag is shown here. *Bracketed items are optional.*

```
<APPLET
  [CODEBASE = URL to an applet]
   CODE = applel class filename
  [ARCHIVES = Name of a JAR file]
  [OBJECT Serialized applet filename]
  [ALT = alternate text]
  [NAME = applet instance name]
   WIDTH = pixels HEIGHT = pixels
  [ALIGN = alignment type]
  [VSPACE = pixels] [HSPACE = pixels]
>
  [< PARAM NAME = AttributeName VALUE = AttributeValue>]
  [< PARAM NAME = AttributeName2 VALUE = AttributeValue>]
  . . .
  [Some text to be displayed in the absence of Applet]
</APPLET>
```

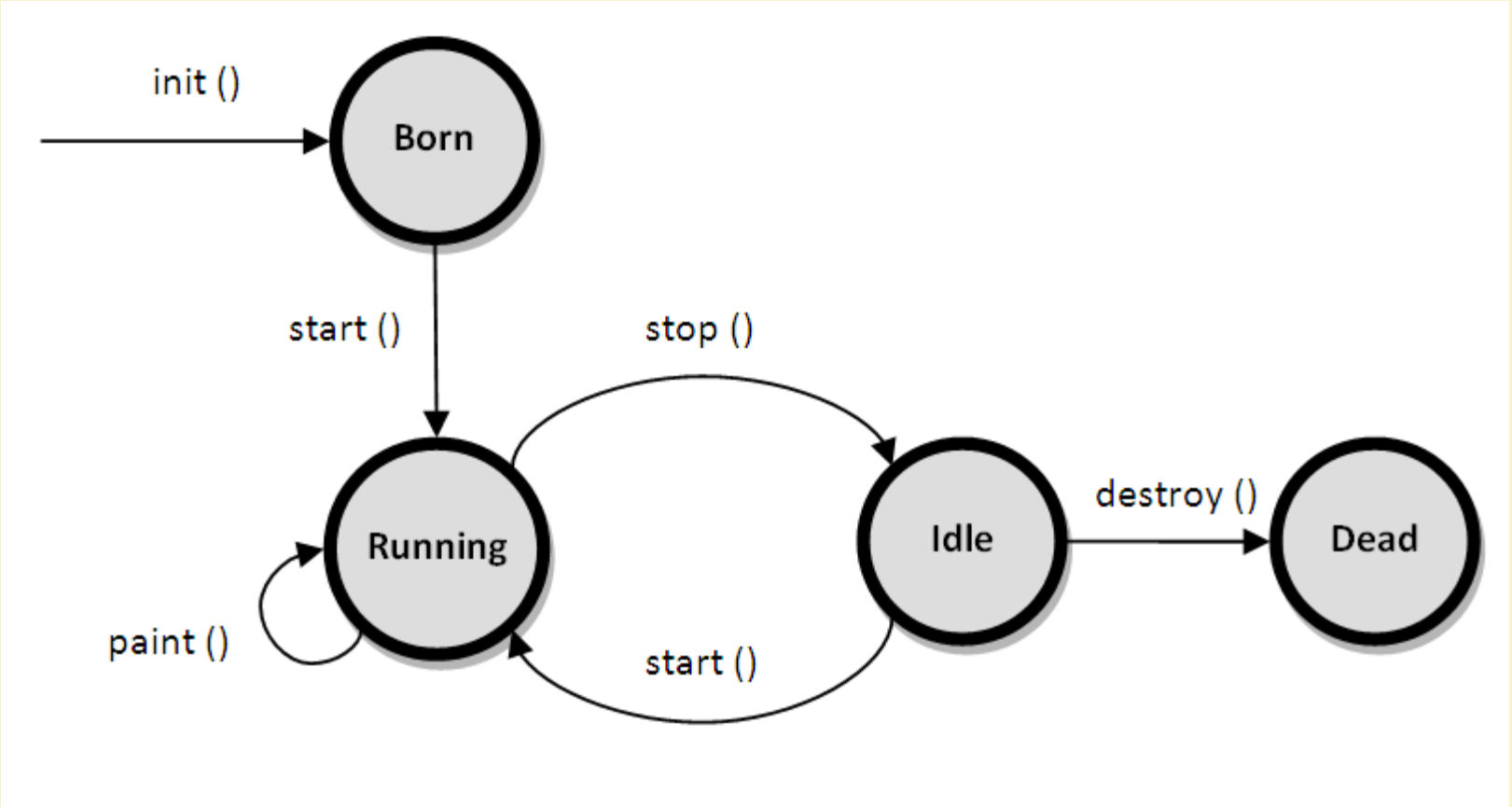
Running an HTML file: An example

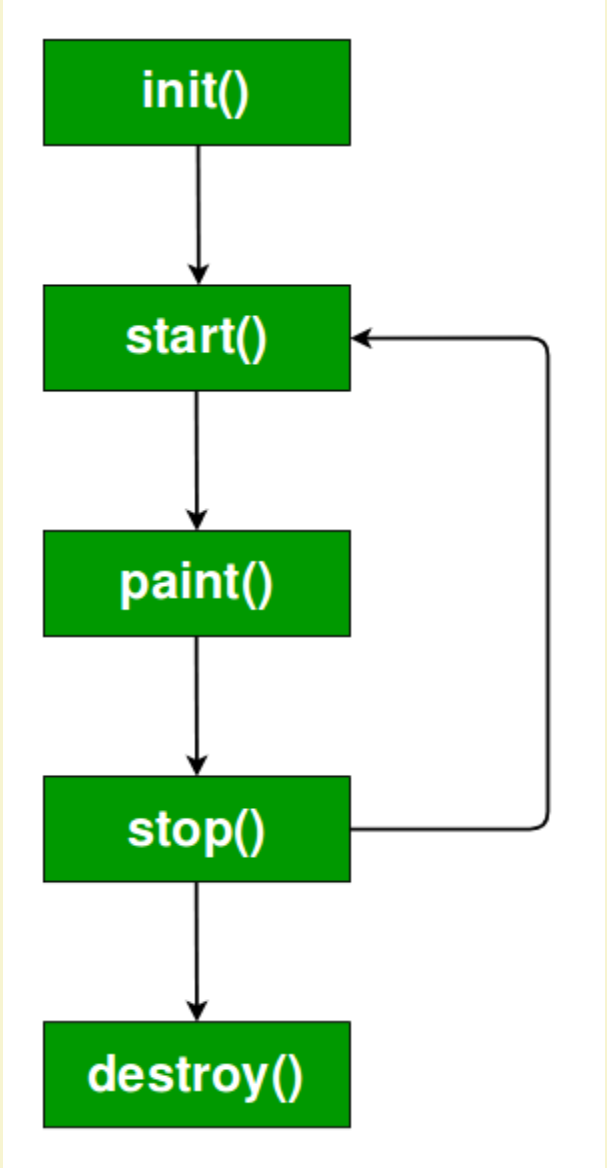
```
import java.awt.*;
import java.applet.*;
public class StatusWindow extends Applet{
  public void init() {
    setBackground(Color.cyan);
  }
  public void paint(Graphics g) {
    g.drawString("This is in the applet window.", 10, 20);
    showStatus("This is shown in the status window.");
  }
/*
<html>
   <body>
      <applet width="300" height="50" code="StatusWindow.class">
      </applet>
   </body>
</html>
*/
```

Tips:

1. Save this file as `StatusWindow.java`
2. Compile it.
3. Run it as : **`appletviewer StatusWindow`**

Applet Life Cycle





Basic Methods of Applet

- public void init() – To initialize or pass input to an applet
- public void start()- start() called after the init() method and it starts an applet
- public void stop()- To stop running applet
- public void paint(Graphics G)- To draw something within an applet
- Public void destroy() – To remove an applet from memory completely

Order of invocation of Applet methods

- These are abstract methods defined in abstract class Applet and they need to be overridden in applet programs. It is important to note the order in which the various methods are called.

- When an applet begins, the AWT calls the method in the sequence

`init()` → `start()` → `paint()`

- When an applet is terminated, the following sequence of methods call takes place

`stop()` → `destroy()`

- Note: All these methods are optional.

Applet initialization and start : Methods

init( )

The `init()` method is the first method to be called. This is where you should initialize an applet. This method is called only once during the run time of your applet.

start( )

The `start()` method is called after `init()`. It is also called to restart an applet after it has been stopped. Whereas `init()` is called once—the first time an applet is loaded—`start()` is called each time an applet's HTML document is displayed onscreen. So, if a user leaves a web page and comes back, the applet resumes execution at `start()`.

Applet paint method

paint( )

The **`paint()`** method is called each time the applet's output to be redrawn. This situation can occur for several reasons. For example, the window in which the applet is running may be overwritten by another window and then uncovered. Or the applet window may be minimized and then restored. **`paint()`** is also called when the applet begins execution. Whatever the cause, whenever the applet must redraw its output, **`paint()`** is called. The **`paint()`** method has one parameter of type **`Graphics`**. This parameter will contain the graphics context, which describes the graphics environment in which the applet is running. This context is used whenever output to the applet is required.

Applet termination : Methods

stop( )

The `stop()` method is called when a web browser leaves the HTML document containing the applet—when it goes to another page, for example. When `stop()` is called, the applet is probably running. You should use `stop()` to suspend threads that do not need to run when the applet is not visible. One can restart them when `stop()` is called if the user returns to the page.

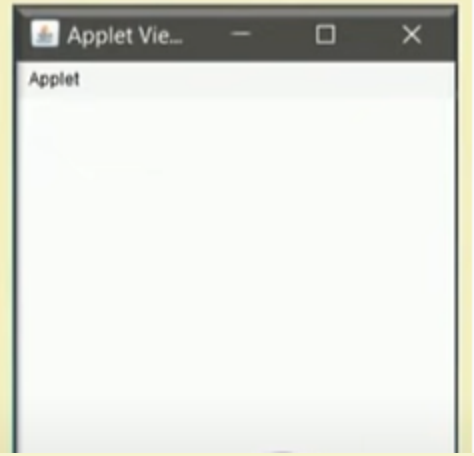
destroy( )

The `destroy()` method is called when the environment determines that your applet needs to be removed completely from memory. At this point, you should free up any resources the applet may be using. The `stop()` method is always called before `destroy()`.

Applet with blank methods

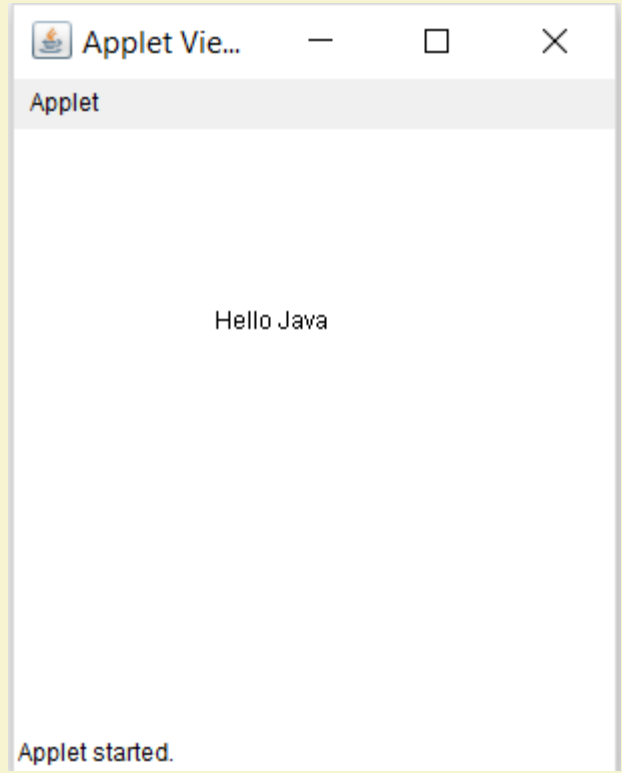
```
import java.awt.*;
import java.applet.*;

public class AppletSkeleton extends Applet {
  public void init() {  }
  public void start() {  }
  public void stop() {  }
  public void destroy() {  }
  public void paint(Graphics g) {  }
}
```

```
<html>
   <body>
      <applet width="300" height="300" code="AppletSkeleton.class">
      </applet>
   </body>
</html>
```



Example :- Print the Hello Java on Applet Window

```
import java.awt.*;
import java.applet.*;
public class HelloJava extends Applet
{
    public void paint(Graphics g)
    {
     g.drawString("Hello Java", 100,100);
    }
}

/*
  <html>
     <body>
        <applet code= "HelloJava.class" width ="300" height="300" >
        </applet>
     </body>
  </html>
*/
```



Commonly used methods of Graphics class:

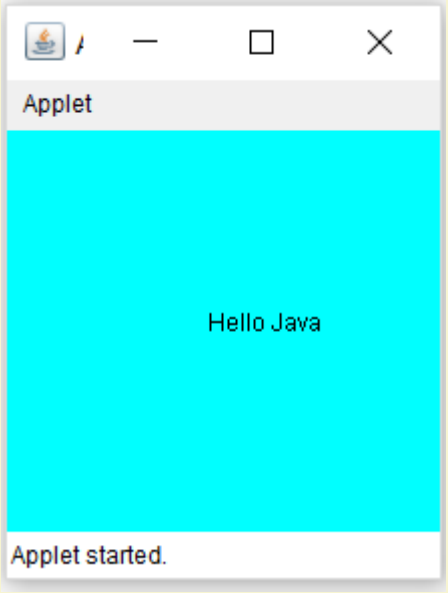
1. **public abstract void drawString(String str, int x, int y):** is used to draw the specified string.
2. **public void drawRect(int x, int y, int width, int height):** draws a rectangle with the specified width and height.
3. **public abstract void fillRect(int x, int y, int width, int height):** is used to fill rectangle with the default color and specified width and height.
4. **public abstract void drawOval(int x, int y, int width, int height):** is used to draw oval with the specified width and height.
5. **public abstract void fillOval(int x, int y, int width, int height):** is used to fill oval with the default color and specified width and height.
6. **public abstract void drawLine(int x1, int y1, int x2, int y2):** is used to draw line between the points(x1, y1) and (x2, y2).
7. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer):** is used draw the specified image.
8. **public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used draw a circular or elliptical arc.
9. **public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used to fill a circular or elliptical arc.
10. **public abstract void setColor(Color c):** is used to set the graphics current color to the specified color.

---

11. **public abstract void setFont(Font font):** is used to set the graphics current font to the specified font.

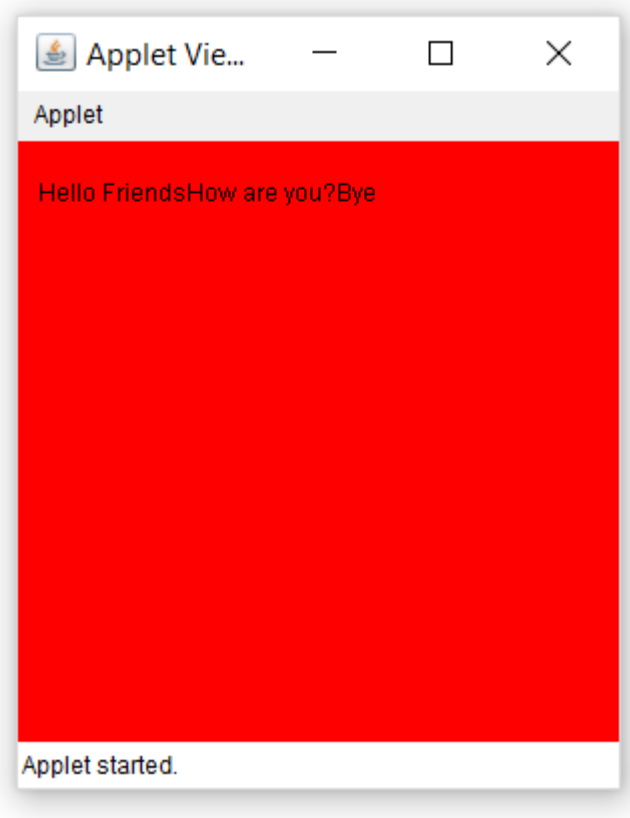
Example

```
import java.awt.*;
import java.applet.*;
public class HelloJava extends Applet
{
   public void init()
   {
    resize(200,200);
    setBackground(Color.cyan);
   }
   public void paint(Graphics g)
   {
     g.drawString("Hello Java", 100,100);
   }
}
/*
  <html>
   <body>
    <applet code= "HelloJava.class" width ="300" height="300" >
</applet>
</body>
</html>
```

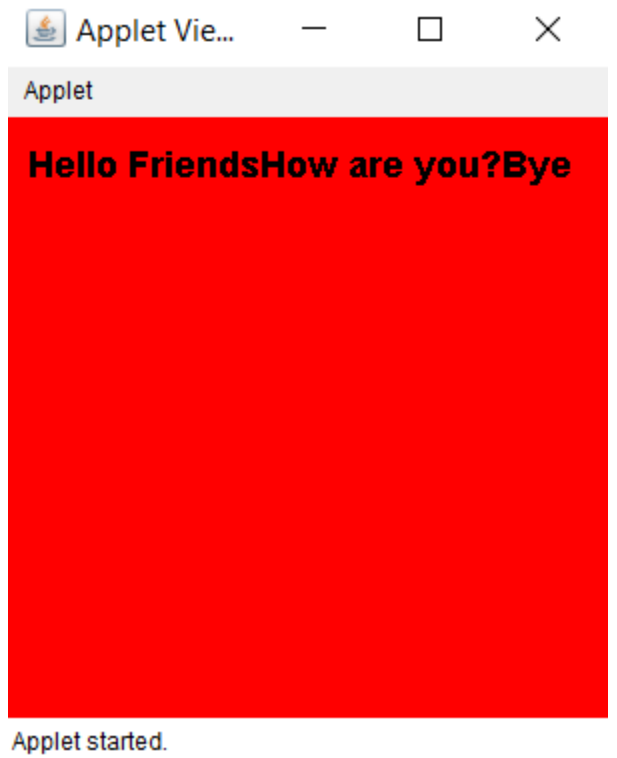


```
import java. applet.*;
import java. awt.*;

public class Sample extends Applet
{
  String msg;
  public void init()
  {
    setBackground(Color.red);   // set background and forground color
                                   of applet window
    setForeground(Color.black);
    msg="Hello Friends …. ";          // initialize the string to dispaly
  }
  public void start()
  {
     msg += "How are you?….";
   }
   public void paint(Graphics g)
   {
     msg+= "Bye…….";
     g.drawString(msg,10,30);
  }
}
```

```
 /*
<html>
 <body>
  <applet code="Sample.class" width="300" height="300">
   </applet>
 </body>
</html>
 */
```



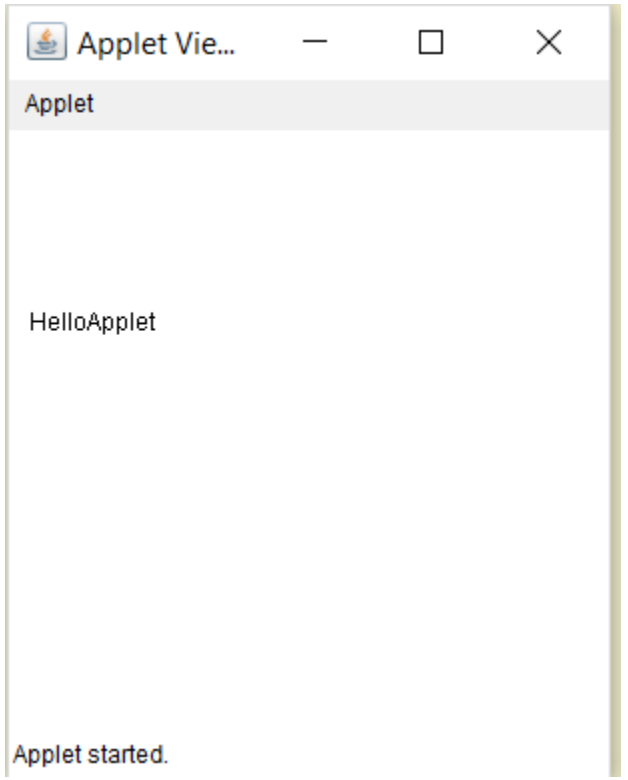
```
import java. applet.*;
import java. awt.*;

public class Sample extends Applet
{
  String msg;
   Font f1;
  public void init()
  {
   setBackground(Color.red);   // set background and forground color
                                  of applet window
   setForeground(Color.black);
   msg="Hello Friends …. ";          // initialize the string to display
   f1 =new Font (“Aerial”,Font.BOLD, 18);
  }
  public void start()
  {
    msg += "How are you?….";
  }
  public void paint(Graphics g)
  {
    msg+= "Bye…….";
    g.setFont(f1);
    g.drawString(msg,10,30);
  }
}


/*
<html>
 <body>
  <applet code="Sample.class" width="300" height="300">
  </applet>
 </body>
</html>
*/
```



Passing a Parameter to Applet

```
import java.applet.Applet;
import java.awt.Graphics;

public class Test extends Applet
{
   String str;
  public void init()
   {

       str=getParameter("string");
       if (str ==null)
         str= "Java";
      str="Hello"+str;
   }
   public void paint(Graphics g)
   {
     g.drawString(str,10,100);
  }
}
```

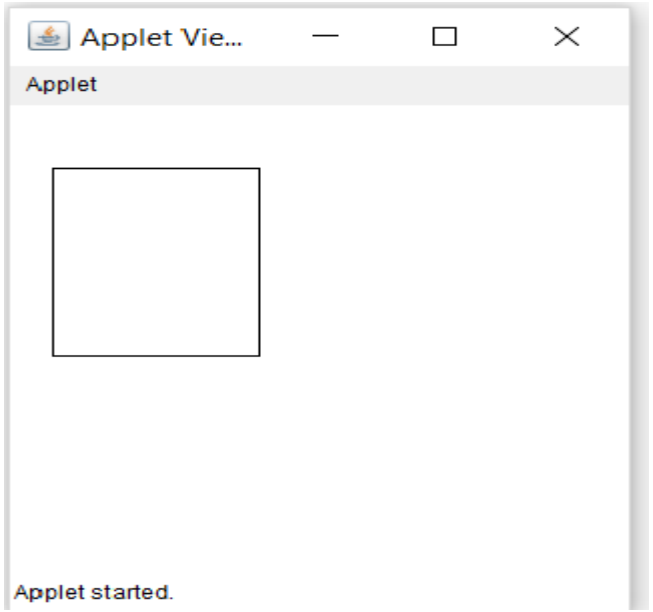
```
/*
<html>
  <body>
    <applet code="Test.class" width="300"
height="300">
  <param name="string" value="Applet">
</applet>
</body>
</html>
*/
```



Passing a Parameter to Applet

```
import java.awt.Graphics;
import java.applet.Applet;

public class Rectangle extends Applet
{
  int x,y,w,h;
  public void init()
  {
    x=Integer.parseInt(getParameter("xValue"));
    y=Integer.parseInt(getParameter("yValue"));
    w=Integer.parseInt(getParameter("wValue"));
    h=Integer.parseInt(getParameter("hValue"));
  }

  public void paint(Graphics g)
  {
    g.drawRect(x,y,w,h);
  }
}
```

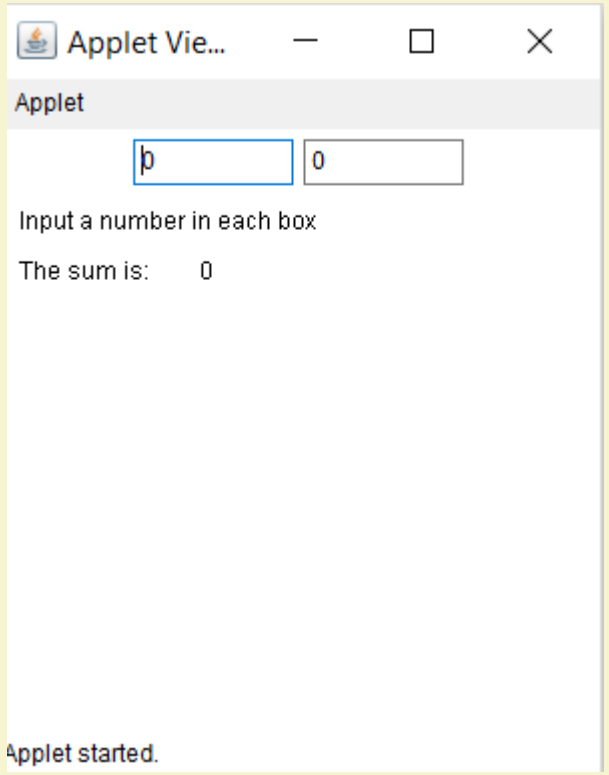
```
/*
<html>
  <body>
    <applet code="Rectangle.class" width="300"
height="300">
    <param name="xValue" value="20">
    <param name="yValue" value="40">
    <param name="wValue" value="100">
    <param name="hValue" value="120">
  </applet>
  </body>
 </html>
 */
```

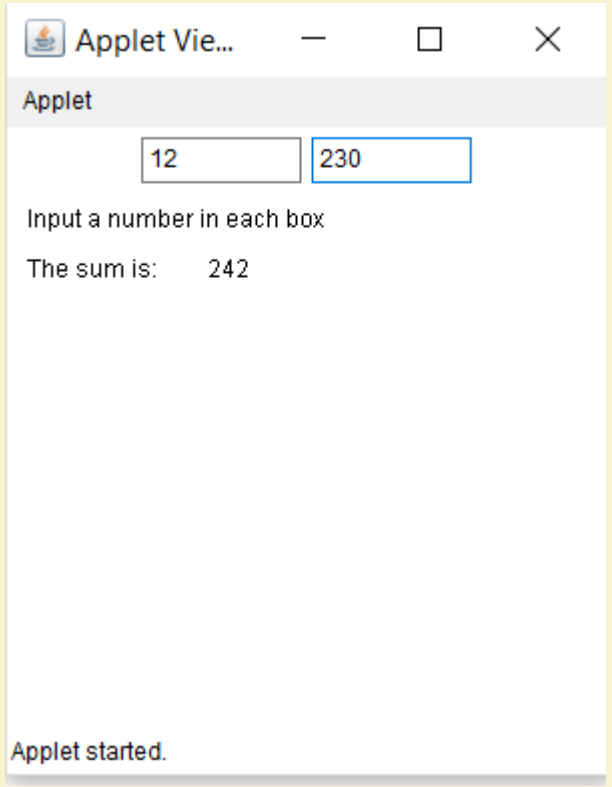


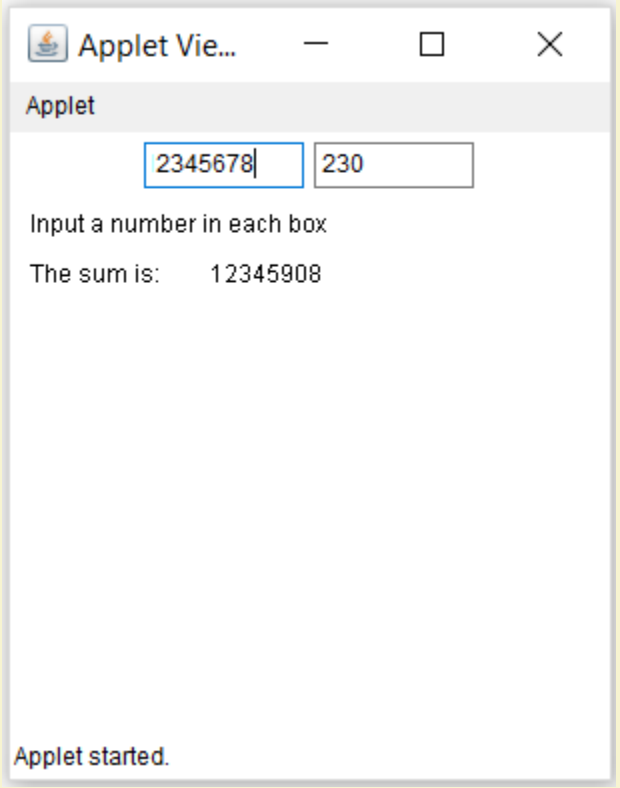
Passing a Parameter to Applet

```
import java.awt.*;
import java.applet.*;

public class UserIn extends Applet
{
   TextField text1, text2;
   public void init()
   {
     text1=new TextField(8);
     text2=new TextField(8);
    add(text1);
    add(text2);
    text1.setText("0");
     text2.setText("0");
  }
public void paint(Graphics g)
{
  int x=0,y=0,z=0;
  String s1,s2,s;
    g.drawString("Input a number in each box",10,50);

    try
    {
      s1=text1.getText();
      x= Integer.parseInt(s1);
      s2=text2.getText();
       y=Integer.parseInt(s2);
     }
   catch(Exception e)
    {}
     z=x+y;
     s= String.valueOf(z);
     g.drawString("The sum is:",10,75);
     g.drawString(s,100,75);
   }

public boolean action (Event event, Object object)
{
   repaint();
   return true;
}
}
/*
<html>
 <body>
  <applet code ="UserIn.class" width="300" height="300">
 </applet.
 </body>
<html>
*/
```

Passing a Parameter to Applet







Unit No.5, Part III Assignment No.

- Create an applet with three text fields and four buttons add, substract, multiply and divide. User will enter two values in the Text Fields. When any button is pressed, the corresponding operation is performed and the results is displayed in the third text fields.

- Assignment Deadline :-

Fundamental of JAVA Programming

UNIT :- V

Exception Handling , Multithreading, Applet

Topic

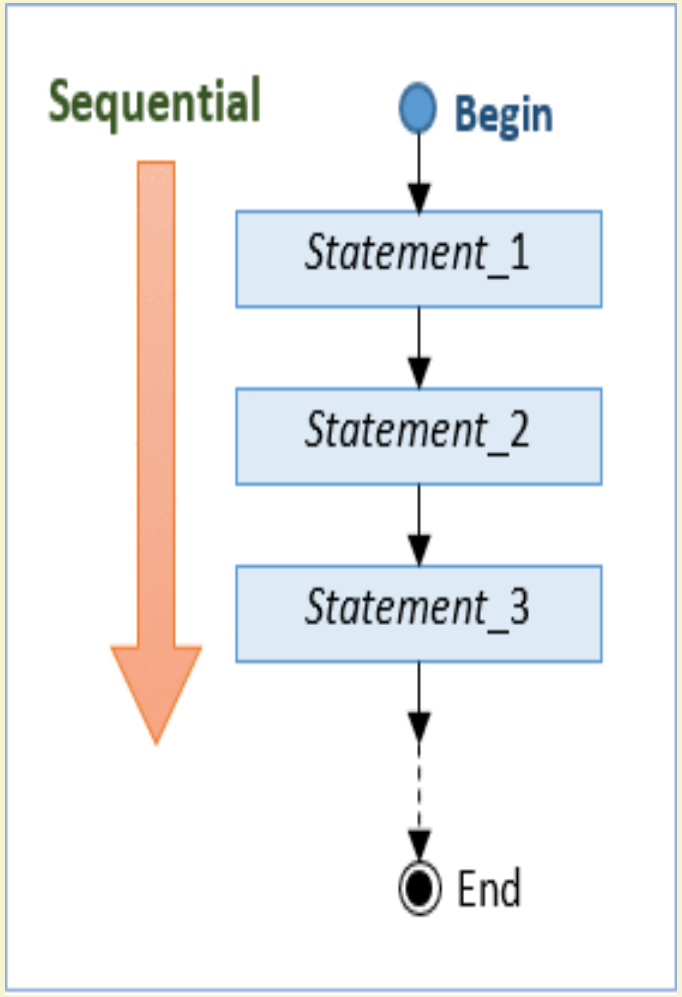
Multithreading

Multiple Threading

- Multitasking :- Whenever system can execute the several task simultaneously, then this ability called as Multitasking
- In programming terminology it is called as multithreading
- Multithreading :- It is a conceptual programming paradigm where a program is divided into two or more subprograms, which can be implemented at the same time in parallel.
- Example:- One subprogram can display an animation on the screen while another may build the next animation to be displayed
- Similar to dividing a task into subtasks and assigning them to different people for execution independently and simultaneously.

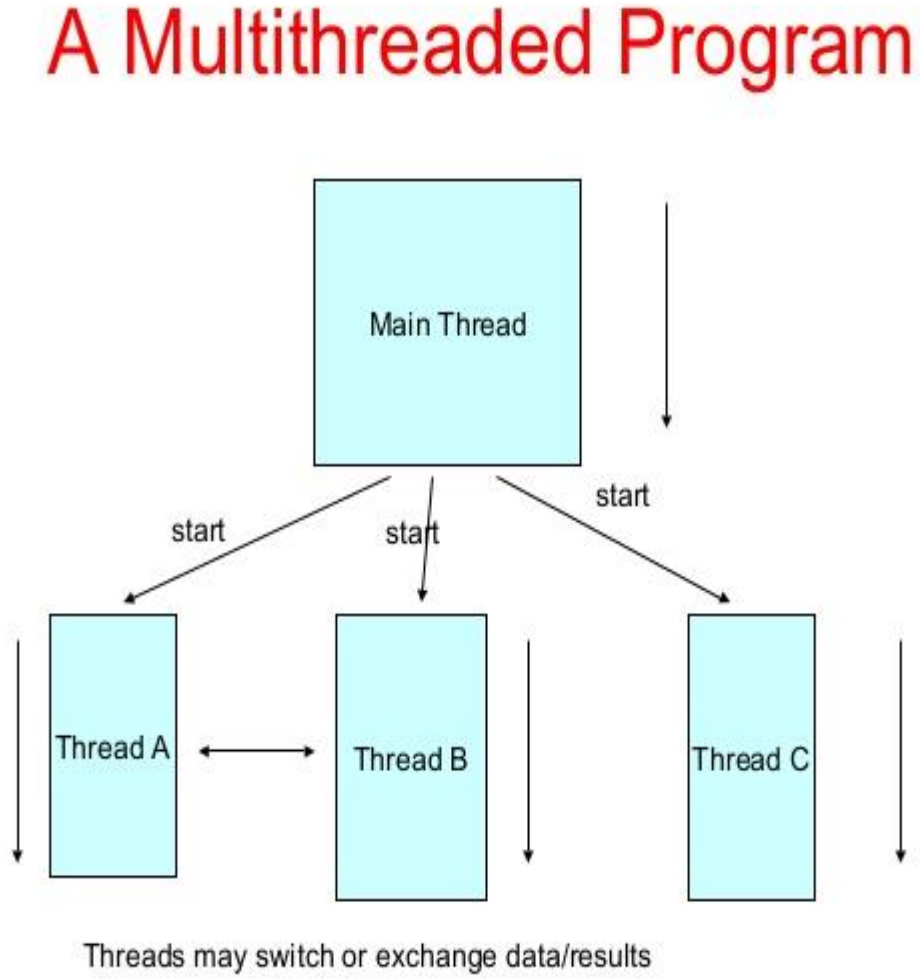
Multiple Threading

- Java programs that we have seen and discussed up to now, it has single sequential flow of control.
- Program begins, runs through a sequence of execution and finally ends.
- At a time, there is only one statement under execution
- Thread is similar to a program that has a single flow of control. (i.e all earlier examples can be called as single threaded programs.
- Every program will have at least one thread



Multiple Threading

- Unique property of java is its support for multithreading.
- Multiple flow of control, each flow of control may be through as a thread that runs in parallel.
- Multithreaded Program :- A program that contain multiple flow of control is known as multithreaded program
- It is important to remember that threads running in parallel does not mean that they actually run at the same time



Creating Threads

- Threads are implemented in the form of objects that contain a method called run().
- Run() method is the heart and soul of thread
- New thread can be created in two ways

1. By Creating a thread class
2. By converting a class to thread

By Creating a Thread Class

- Define a class thread that extends Thread class and override its run() method with the code required by the thread
- Process-1) Creating new thread class by using extends the Thread class which is mention in the lang package
- Process -2) Override the run() method which is mention the Thread class and implement the thread code
- Process-3) Create the object for new thread class
- Process -4) Invoked the run() method by using object of new thread class and start() method
- By using start method, we have to start the execution of run() method.

```
import java.lang.Thread;
class Mythread extends Thread
{
     public void run()
     {
       ----------
       ----------
     }
}

class Main
{
 public static void main(String args [])
{
    Mythread obj=new Mythread();
     obj.start();
 }
}
```

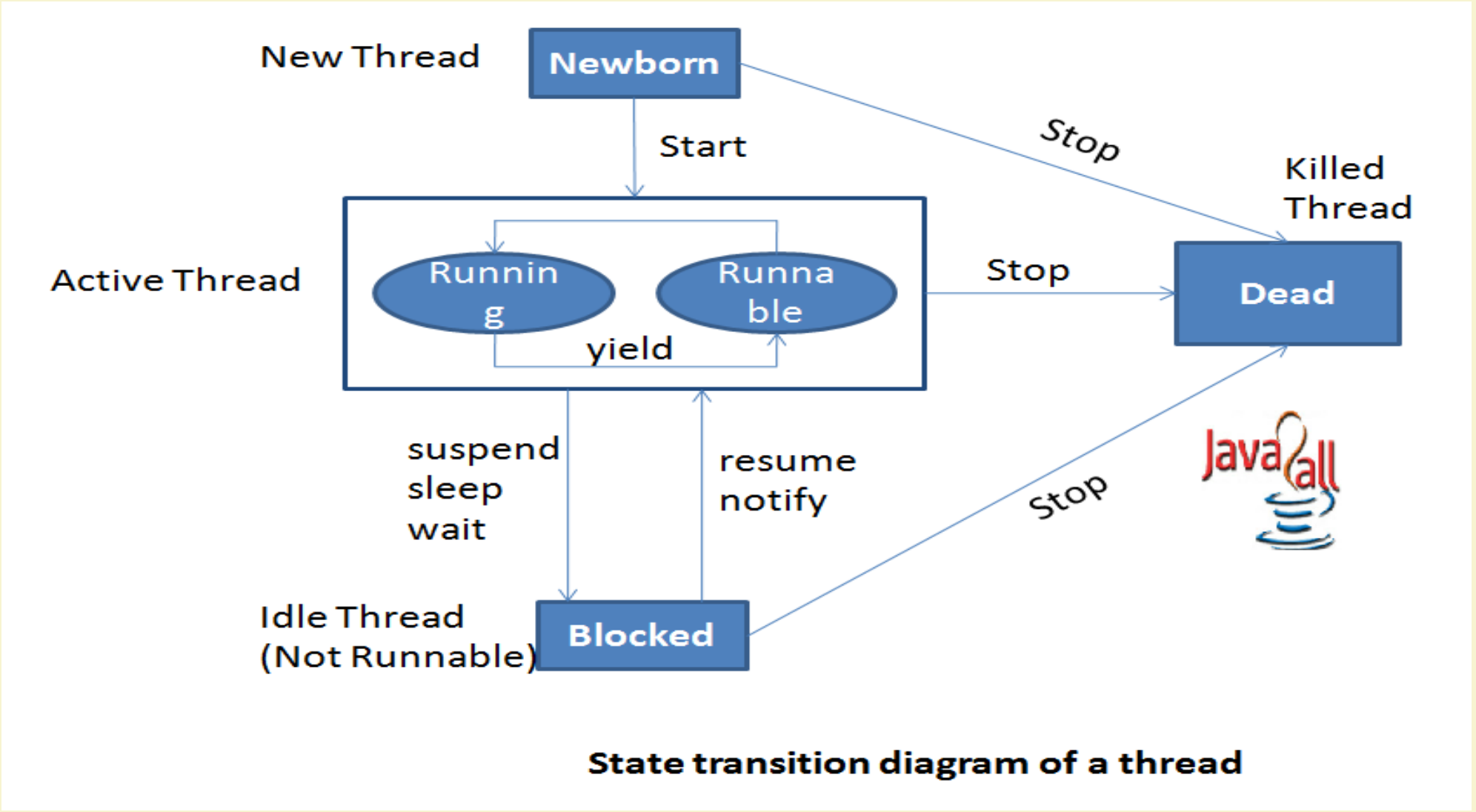
Stopping & Blocking Thread

- Stopping Thread:- Whenever we want to stop a thread from running further, we may calling stop() method
- e.g. aThread.stop();
- Blocking Thread :- A thread can also be temporarily suspended or blocked from entering into runnable and subsequently running state by using either of the following methods

```
sleep()          //Blocked for a specified time
suspended()      //Blocked until further orders
wait()           // Blocked until certain condition occurs
```

- The thread will return to the runnable state when the specified time is elapsed in the case of sleep(), the resume() method is invoked in the case of suspend(), and the notify() method is called in the case of wait().

Life Cycle of Thread

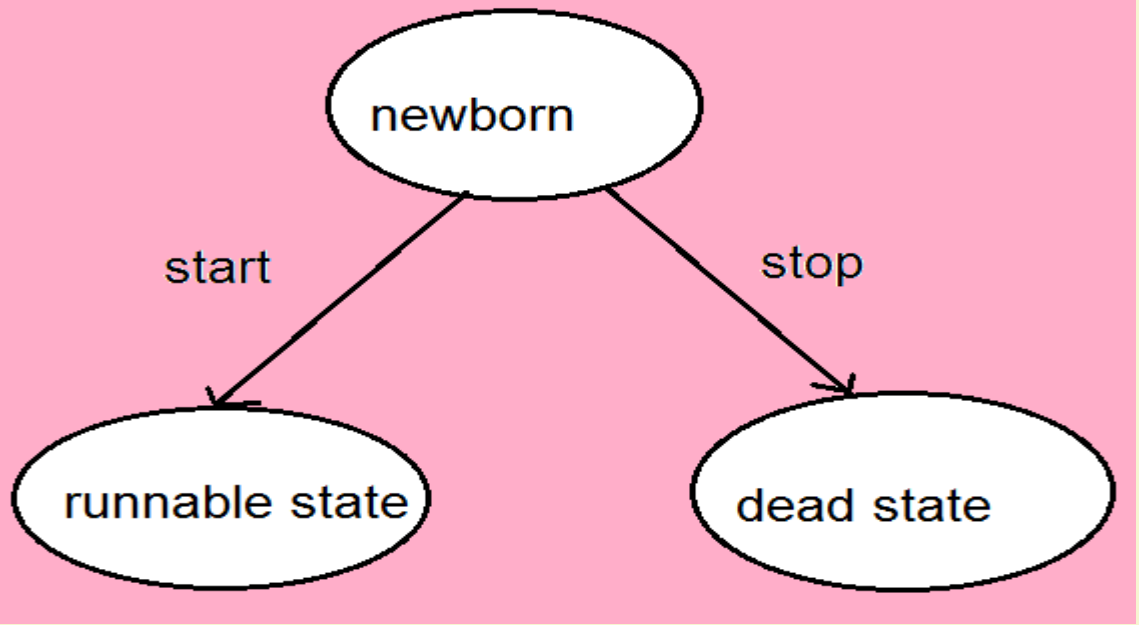


State transition diagram of a thread

Life Cycle of Thread

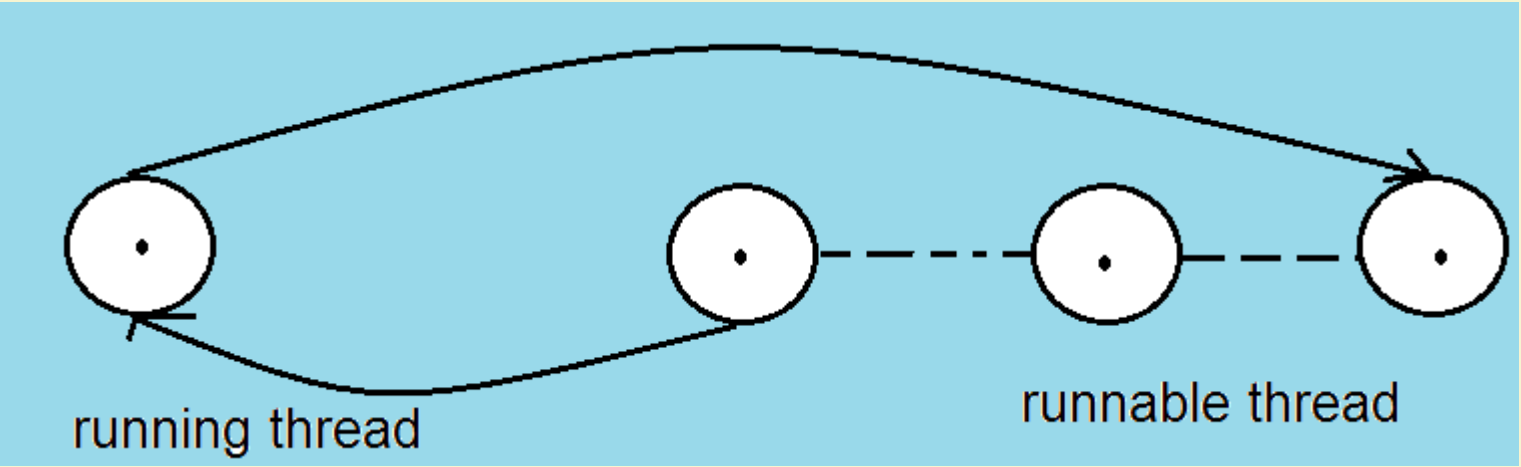
- Newborn State :- when we create a thread object the thread is born and is said to be in newborn state.
- Thread is not yet scheduled for running
- At this state, we can do only one of the following things with it

1. Schedule it for running using start() method
2. Kill it using stop() method



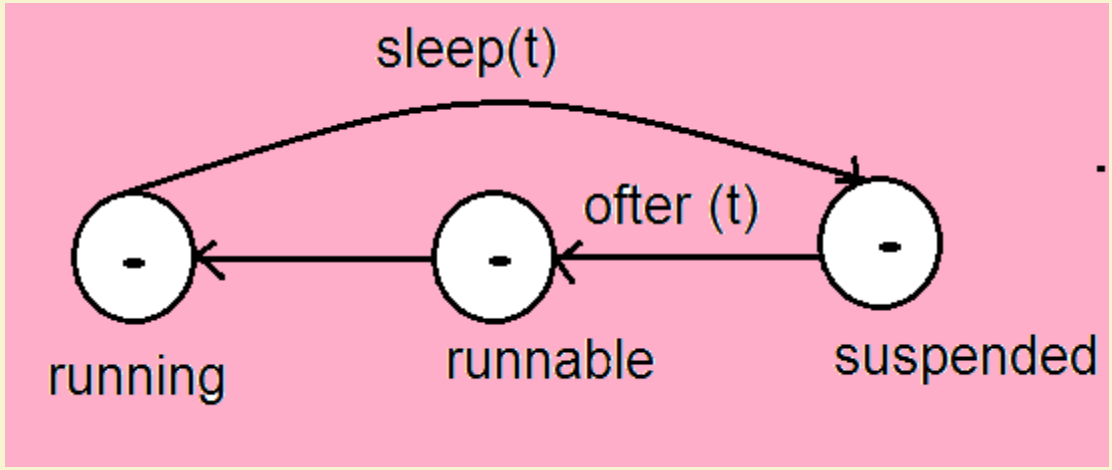
Life Cycle of Thread

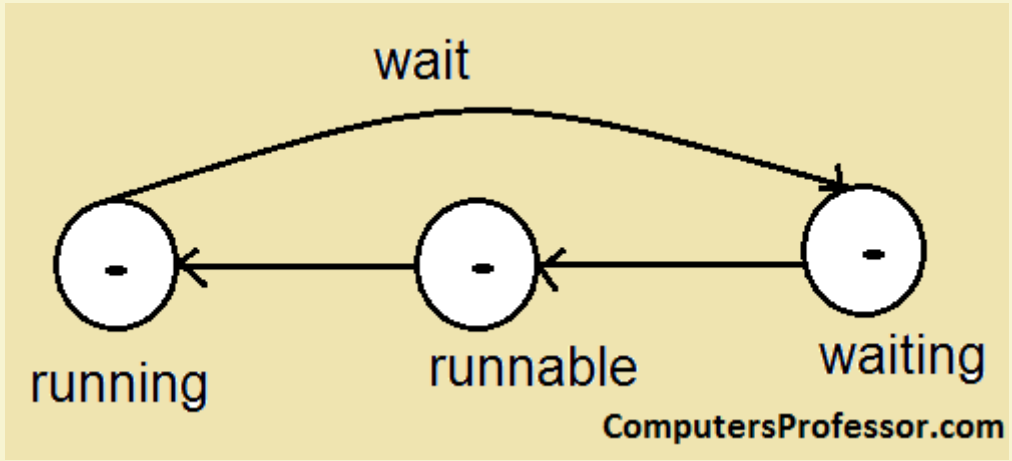
- Runnable State :- Runnable state means that the thread is ready for execution and is waiting for the availability of the processor
- That means thread has joined the queue of threads that are waiting for execution



Life Cycle of Thread

- Running State :- Running state means that the processor has given its time to the thread for its execution
-





Thread Priority

- Thread Priority :-
- In Java, each thread is assigned a priority, which affects the order in which it is scheduled for running.
- Java permits us to set the priority of a thread using the setPriority() method as follows
- ThreadName.setPriority(intNumber);
- The intNumber is an integer value to which the threads priority set.
- intNumber is any value between 1 to 10
- Note that default setting is NORM_PRIORITY

MIN_PRIORITY =1

NORM_PRIORITY =5

MAX_PRIORITY =10